General Coding Principles

1.1 Readability

- Code should be easy to read and understand.
- Prefer clear and simple implementations over unnecessarily complex solutions.
- Avoid writing multiple unrelated operations in a single statement.
- Use meaningful names for variables, functions, classes, and modules.
- Code should be understandable to another team member without requiring extensive explanation.

1.2 Modularity

- Each module should have a clearly defined responsibility.
- Functions and classes should perform one primary task.
- Avoid creating large functions containing unrelated operations.
- Common functionality should be placed in reusable modules rather than duplicated.
- Modules should interact through clearly defined interfaces.

1.3 Maintainability

- Code should be written with future modifications in mind.
- Avoid hard-coded values when configuration or constants are more appropriate.
- Changes to one feature should cause minimal changes to unrelated modules.
- Before adding a new feature, existing functionality should be checked to avoid duplication.

1.4 Reliability

- Operations that may fail should include appropriate error handling.
- External dependencies such as APIs, Windows services, files, and processes should not be assumed to always be available.
- The system should provide meaningful error messages rather than silently failing.
- Critical operations should be verified after execution whenever practical.

Avoid Unnecessary Automation Complexity

For Windows operations, the simplest reliable execution method should be preferred.

The preferred order is:

1. Direct Python or Windows API
2. PowerShell or command-line interface
3. Windows UI Automation
4. GUI-based automation
5. Screen/image-based automation as a fallback

Screen-coordinate-based automation should not be used when a more reliable interface is available.

Python Coding Standards

2.1 Code Formatting

- Follow **PEP 8** as the primary Python style guideline.
- Use consistent indentation of **4 spaces**.
- Do not use tabs for indentation.
- Keep lines reasonably short and readable.
- Use blank lines to separate logical sections of c

2.2 Imports

Imports should be organized into:

1. Standard library imports
2. Third-party libraries
3. Project-specific imports

Example:

```
import os
import subprocess

import psutil
from PySide6.QtWidgets import QApplication

from core.executor import Executor
from utils.logger import logger
```

Unused imports should be removed.

2.3 Functions

- Functions should be focused on a single responsibility.
- Function names should clearly describe their action.
- Functions should generally avoid excessive numbers of parameters.
- Return values should be predictable and documented when necessary.

Example:

```
def create_folder(path):
    ...
```

is preferred over:

```
def process(path, mode, flag, option):
    ...
```

when the function is specifically responsible for creating a folder.

2.4 Classes

- Classes should represent a clear concept or responsibility.
- Avoid creating classes simply to contain unrelated functions.
- Large classes should be divided when they begin handling multiple independent responsibilities.

Example:

```
class FileManager:
    ...
```

is preferable to a general-purpose class such as:

```
class SystemManager:
    # Files
    # Processes
    # Network
    # GUI
    # Database
```

2.6 Comments

Comments should explain **why** something is done when the reason is not obvious.

Avoid comments that simply repeat the code.

Bad:

```
# Increment count
count += 1
```

Better:

```
# Retry only temporary Windows process failures.
count += 1
```

2.7 Type Hints

Type hints should be used for important functions and public interfaces.

Example:

```
def create_folder(path: str) -> bool:
    ...
```

This is especially useful for communication between the core system and plugins.

2.8 Constants

Values that are reused or represent configuration should not be scattered throughout the code.

Example:

```
MAX_RETRIES = 3
DEFAULT_TIMEOUT = 10
```

rather than repeatedly using unexplained numbers.

Naming Conventions

Consistent naming is required throughout the project.

3.1 Variables

Use `snake_case`.

```
file_path
process_name
command_result
```

Avoid:

```
filePath
ProcessName
f
```

unless a very short local variable is appropriate.

3.2 Functions

Use `snake_case` and start with a descriptive verb where appropriate.

```
create_folder()
open_application()
get_system_info()
execute_command()
```

3.3 Classes

Use `PascalCase`.

```
FileManager
CommandParser
ExecutionEngine
PluginManager
SystemMonitor
```

3.4 Constants

Use uppercase `SNAKE_CASE`.

```
MAX_RETRIES
DEFAULT_TIMEOUT
PLUGIN_DIRECTORY
```

3.5 Modules and Files

Use lowercase `snake_case`.

```
command_parser.py
execution_engine.py
file_manager.py
system_monitor.py
```

3.6 Boolean Variables

Boolean names should clearly indicate a true/false condition.

Preferred:

```
is_running
is_enabled
has_permission
file_exists
```

Avoid ambiguous names such as:

```
status
flag
value
```

when they represent a boolean.

3.7 Plugin Names

Plugin names should clearly identify the functionality they provide.

Examples:

```
file_manager
system_utils
process_manager
browser_plugin
```

Avoid generic names such as:

```
plugin1
test_plugin
manager
helper
```

unless they are genuinely appropriate.

4.1 Core

Contains the main logic of the application.

Examples:

- Command processing
- Task planning
- Execution management
- Tool/feature registration

The core should not contain application-specific implementation when that functionality belongs in a plugin.

4.2 Plugins

Each plugin should provide a specific group of capabilities.

For example:

```
plugins/
└── file_manager/
    ├── plugin.py
    ├── operations.py
    └── tests/
```

Plugins should communicate with the core through defined interfaces rather than directly modifying core components.

4.3 GUI

Contains only presentation and user-interaction logic.

The GUI should not directly implement file operations, process management, or AI logic.

For example:

```
GUI
 ↓
Core
 ↓
Plugin
 ↓
Windows Operation
```

rather than:

```
GUI
 ↓
os.remove(...)
```

4.4 Tests

Tests should be separated from production code.

At minimum:

```
tests/
├── unit/
├── integration/
└── end_to_end/
```

- **Unit tests:** Test individual functions or classes.
- **Integration tests:** Test interaction between modules.
- **End-to-end tests:** Test complete user workflows.

4.5 Documentation

Project documentation should be stored separately from source code.

Examples:

```
docs/
├── architecture.md
├── plugin_guide.md
├── coding_standards.md
└── testing.md
```

The coding standards document itself should therefore live under `docs/`.